

AWS Certified Generative AI Developer — Professional

The Use Case

- Healthcare assistant on Bedrock — patient information, medical literature, treatment guidelines, serving thousands of providers across hospitals and clinics

Four reported problems (these structure the whole task):

- Incomplete responses on complex patient histories
- API failures during peak usage
- Inconsistent responses to similar medical queries
- Slow or irrelevant retrieval on specialised topics

Task	Nature
5.1	Evaluation — how good is it? (proactive, systematic)
5.2	Troubleshooting — why is it broken? (reactive, diagnostic)

1. Content Handling — Incomplete Responses

Diagnosis: context window overflow.

Diagnose first

```
CloudWatch metrics -> token usage patterns
custom dashboards per query type
-> patient history queries were approaching the
    model's 8,000 token limit
```

Also analyse **token distribution across document types** — not everything overflows, one specific type does. Fix that.

Dynamic chunking

```
from fixed-size to SEMANTIC chunking
keep related medical information together
using natural boundaries:
    visit dates
    treatment episodes
```

"Dynamic" means segmentation adapts to **content complexity and semantic boundaries** rather than a fixed size. This is Task 1.5's structure-aware chunking in a medical domain.

Prompt design and truncation detection

```
restructure how history is requested
-> recent and clinically significant information FIRST
-> pagination for extensive histories
```

```
CloudWatch Logs analysis
-> truncation patterns found in MEDICATION LISTS
    and TREATMENT HISTORIES
-> specialised handling for these high-value fields
```

Truncation is silent. The model answers from whatever it received, and the answer reads confidently. This is why the notes call for **alerts when important content is being cut off** rather than relying on someone noticing.

Sliding window and supporting techniques

```
Step Functions sliding window
-> process large records in manageable segments
-> MAINTAINING clinical context between chunks
```

- Prompt optimisation framework — reduce tokens while preserving critical information
- Specialised preprocessing per content type (technical documentation, conversational text, code)
- Context prioritisation algorithms** keep the most relevant material inside the window
- Automated testing across document sizes and types; visualisation of token usage patterns

2. FM Integration — API Failures

Structured logging

```

categorise errors in CloudWatch by
  TYPE / SEVERITY / FREQUENCY
-> revealed throttling errors during MORNING ROUNDS,
    when usage spikes

```

"API failed" tells you nothing. Only categorised logging distinguishes throttling from timeout from validation failure — the 429 / 500 / 400 distinction from Task 2.4.

Request validation

```

Lambda catches malformed requests
  BEFORE they reach the Bedrock API
-> error rates down 40%

```

Nearly half the "API failures" were the application's own malformed requests. Validating before the call removed them entirely.

Response analysis

Detect patterns in model outputs that indicate integration problems — incomplete responses, unexpected formatting. **An API can return success while the output is still wrong;** that is also an integration issue.

X-Ray end-to-end tracing

```

bottleneck found in the AUTHENTICATION MIDDLEWARE,
  which was contributing to timeouts

```

The problem was not in the model. Without end-to-end tracing the team would have spent weeks optimising inference and changed nothing.

Circuit breakers and load shedding

```

Step Functions
-> handle API rate limits gracefully
-> LOAD SHEDDING: deliberately drop some requests
    at peak so the rest continue
-> CRITICAL queries prioritised

```

In healthcare: emergency queries proceed, routine lookups wait. Supporting practices: automated connectivity testing, CloudWatch dashboards for error rates, specialised handlers per error type, alerting on unusual patterns, retry with exponential backoff.

3. Prompt Engineering — Inconsistent Responses

This is **context sensitivity** — the GenAI-specific failure mode identified in Task 4.3.

- **Systematic prompt testing (Step Functions)** across medical specialties and query types — one prompt does not serve every domain
- **Version control with rollback** when performance degrades (Task 1.6)
- **A/B testing via Bedrock Prompt Management**, measuring accuracy *and consistency* — consistency is explicitly measured because it is the presenting problem
- **CoT analysis** to find reasoning failures in complex decision-support prompts — improved diagnostic suggestion accuracy. (Task 4.3's reasoning path tracing, used diagnostically.)

Prompt complexity metrics

A counter-intuitive point: the notes call for metrics that identify **overly complicated instructions that confuse the model**. More instructions do not mean better answers — conflicting or deeply nested instructions degrade output.

Plus prompt regression testing, specialised templates per task type, automated optimisation suggestions from performance data, and refinement workflows fed by user feedback.

4. Retrieval System — Slow or Irrelevant Results

Relevance and embedding diagnostics

- **Relevance scoring** via Bedrock Evaluations — how well retrieved literature matches physician queries
- **Embedding visualisation (SageMaker)** to spot **clustering issues and outliers** in the medical terminology vector space. Poor clustering means **the embedding model doesn't understand the domain** — the diagnostic counterpart to Task 1.5's point that domain fit matters more than dimensionality.
- **Drift monitoring (SageMaker Model Monitor)** for embeddings diverging from expected patterns **as new research and terminology appear** — medical language keeps changing, and old embeddings don't know new terms

ANN algorithm selection

Algorithm	Characteristics
LSH — Locality-Sensitive Hashing	Hash-based; fast, less accurate
HNSW	Graph-based; fast and accurate, memory-hungry
IVF — Inverted File Index	Cluster-based; lower memory, good at scale
PQ — Product Quantization	Compression-based; much less memory, some accuracy loss

Trade-offs: accuracy vs speed vs memory vs scalability

"Benchmark multiple algorithms with YOUR specific dataset." There is no universally best choice — the answer depends on your data, and must be measured.

Specialised chunking and RAG pipeline logging

medical literature chunking must preserve:
research findings + methodology + conclusions
(separated, a finding without its methodology invites misinterpretation)

Track and log EVERY RAG pipeline step:
data preparation
chunking (strategy, chunk size, OVERLAP, chunk COUNTS)
ingestion
retrieval
evaluation

Also record:
embedding model, vector database details,
document ingestion metrics,
retrieval model, CONTEXT SIZE, performance metrics,
evaluation metrics (answer similarity, correctness, relevance)

Additional diagnostics: **vector database health checks** (index corruption, performance), **chunking quality assessment**, **preprocessing validation** (inconsistent normalisation produces inconsistent embeddings), **hybrid search diagnostics**, and feedback loops from user interactions.

Retrieval problems can also be addressed from the prompt side: **query expansion** with AI-generated context, shifting from **fuzzy matching to semantic similarity**, and **hybrid search** combining semantic with traditional retrieval.

5. Prompt Maintenance

Section	Concern
Section 3	The prompt is broken — fix it (reactive)
Section 5	Keeping prompts healthy (ongoing)

Template testing and prompt confusion

- **Template testing frameworks** across scenarios and edge cases — what if a variable is empty, enormous, or contains special characters?
- **CloudWatch Logs Insights queries to identify patterns of prompt confusion:**

```
the model treats an instruction as a question
the answer discusses the template rather than the content
placeholder text ({{variable}}) appears in the output
the model comments on conflicting instructions
```

These patterns are visible in logs — **if you go looking for them.**

X-Ray on the prompt pipeline

Trace the **template rendering process** — fetching the template, substituting variables, assembling context. **This is not the model call**, and it can be slow or fail on its own.

Schema validation and version management

- **JSON Schema validation** for format inconsistencies in **both templates and responses** — here, medical terminology and procedure descriptions
- **Version management tracks performance metrics alongside versions**, not just version numbers:

```
v6 -> accuracy 91%
v7 -> accuracy 94%
v8 -> accuracy 88%    -> roll back
```

- **Automated testing triggered on template modification** — CI/CD applied to prompts
- **Prompt governance** for consistency and quality **across multiple applications** (Task 1.6)
- **Scheduled refinement cycles** driven by physician feedback and performance metrics — proactive rather than reactive

Troubleshooting Methodology — Symptom to Diagnosis

Symptom	Look at	Tool
Incomplete responses	Token usage against context limits	CloudWatch metrics
API failures	Error type, severity, frequency	Structured logging
Slow responses	Where the time is spent	X-Ray tracing
Inconsistent answers	Prompt versions and phrasing variations	A/B testing, output diffing
Irrelevant retrieval	Embeddings and chunking	Embedding visualisation
Faulty reasoning	Step-by-step logic	CoT analysis

The lesson worth carrying

```
Finding 1: the bottleneck was in AUTH MIDDLEWARE, not the model
Finding 2: 40% of "API failures" were the app's OWN
          malformed requests
```

Twice, the problem was not where it appeared to be. That is precisely why end-to-end tracing and structured logging exist — without them you optimise the wrong component and nothing improves.

Reported outcomes

- **95%** reduction in incomplete responses on complex patient histories
- **99.9%** API availability even at peak
- Consistent responses across similar queries; **60%** faster retrieval
- Continuous improvement driven by physician feedback